

Q-Learning Explained

Learning Values Instead of Directly Learning a Policy

1 Goal of This Note

Q-learning is a value-based reinforcement learning algorithm. Instead of directly learning a policy $\pi(a \mid s)$, it learns how good each action is in each state:

$$Q(s, a) = \text{expected future reward after taking action } a \text{ in state } s.$$

Once we know $Q(s, a)$, choosing actions is simple:

$$a^*(s) = \arg \max_a Q(s, a).$$

2 Policy Learning vs Value Learning

In policy gradient, we learn:

$$\pi_\theta(a \mid s).$$

This directly tells us the probability of each action.

In Q-learning, we learn:

$$Q(s, a).$$

This tells us how valuable each action is. The policy is then derived from the Q-values:

$$\pi(s) = \arg \max_a Q(s, a).$$

3 The Bellman Idea

The central idea is recursive. The value of an action equals immediate reward plus future value:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a').$$

This equation says:

value now = reward now + discounted best value later.

4 Temporal Difference Target

When the agent takes action a in state s , it observes:

$$(s, a, r, s').$$

The Q-learning target is:

$$y = r + \gamma \max_{a'} Q(s', a').$$

The prediction is:

$$Q(s, a).$$

The temporal difference error is:

$$\delta = y - Q(s, a).$$

The word temporal means across time. The estimate at time t is improved using the estimate at time $t + 1$.

5 Tabular Q-Learning Update

If the state space is small, we can store Q-values in a table. The update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

Here:

- α is the learning rate.
- γ is the discount factor.
- $r + \gamma \max_{a'} Q(s', a')$ is the target.
- $Q(s, a)$ is the current prediction.

6 A Tiny Table Example

Suppose:

$$Q(s, a) = 2.0, \quad r = 1.0, \quad \gamma = 0.9, \quad \max_{a'} Q(s', a') = 4.0.$$

Then:

$$y = 1.0 + 0.9 \cdot 4.0 = 4.6.$$

The TD error is:

$$\delta = 4.6 - 2.0 = 2.6.$$

If $\alpha = 0.1$, then:

$$Q(s, a) \leftarrow 2.0 + 0.1 \cdot 2.6 = 2.26.$$

The value moved a little toward the target.

7 Minimal Tabular Code

```
from collections import defaultdict
import random

Q = defaultdict(lambda: [0.0, 0.0]) # two actions

alpha = 0.1
gamma = 0.99
epsilon = 0.1

def choose_action(state):
    if random.random() < epsilon:
        return random.choice([0, 1])
    return max(range(2), key=lambda a: Q[state][a])

def update(state, action, reward, next_state, done):
    best_next = 0.0 if done else max(Q[next_state])
    target = reward + gamma * best_next
    error = target - Q[state][action]
    Q[state][action] += alpha * error
```

8 Exploration: Epsilon-Greedy

If the agent always chooses:

$$\arg \max_a Q(s, a),$$

it may never discover better actions.

Epsilon-greedy exploration solves this by choosing:

$$\begin{cases} \text{random action} & \text{with probability } \epsilon, \\ \text{best known action} & \text{with probability } 1 - \epsilon. \end{cases}$$

At the beginning of training, ϵ is often high. Later, it is reduced.

```
epsilon = max(0.05, epsilon * 0.995)
```

9 Why Q-Learning is Off-Policy

Q-learning learns the value of the greedy policy:

$$\max_{a'} Q(s', a').$$

But the agent may collect data using epsilon-greedy exploration.

So the behavior policy is exploratory, but the learned target is greedy. This is why Q-learning is called off-policy.

10 Q-Learning with Neural Networks

For large or continuous state spaces, a table is impossible. CartPole has continuous observations:

$$s = \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix}.$$

We cannot make a table for every possible real-valued state. So we approximate:

$$Q_{\theta}(s, a)$$

with a neural network.

For CartPole, the network can map:

$$\mathbb{R}^4 \rightarrow \mathbb{R}^2.$$

The two outputs are:

$$\begin{bmatrix} Q(s, \text{left}) \\ Q(s, \text{right}) \end{bmatrix}.$$

11 DQN Loss

Deep Q-Networks, or DQN, train a neural network to predict Q-values.

The prediction is:

$$Q_{\theta}(s, a).$$

The target is:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a').$$

The symbol θ^- means a target network. It is a slowly updated copy of the Q-network.

The loss is:

$$\mathcal{L}(\theta) = (Q_{\theta}(s, a) - y)^2.$$

12 Minimal DQN Target Code

```
import torch
import torch.nn.functional as F

q_values = q_net(states) # shape: [batch, actions]
chosen_q = q_values.gather(1, actions[:, None]) # Q(s, a)
```

```

with torch.no_grad():
    next_q = target_net(next_states).max(dim=1).values
    target = rewards + gamma * (1 - dones) * next_q

loss = F.mse_loss(chosen_q.squeeze(), target)

optimizer.zero_grad()
loss.backward()
optimizer.step()

```

13 Experience Replay

DQN usually stores transitions in a replay buffer:

$$(s, a, r, s', d).$$

Then it samples random minibatches from the buffer. This helps because consecutive environment steps are highly correlated.

```

replay_buffer.append((state, action, reward, next_state, done))
batch = random.sample(replay_buffer, batch_size)

```

Replay makes learning more stable and data efficient.

14 Q-Learning vs REINFORCE

Question	REINFORCE	Q-Learning
What is learned?	Policy probabilities	Action values
Core object	$\pi(a \mid s)$	$Q(s, a)$
Action choice	Sample from policy	Argmax Q-value
Update signal	Return G_t	TD target
Wait until episode end?	Usually yes	No
On/off-policy	On-policy	Off-policy

15 Important Intuition

Q-learning does not ask:

Which action did my policy sample, and how good was the episode?

It asks:

What should the value of this state-action pair be, given the reward and the best future action?

So the basic learning signal is:

prediction error between current Q-value and Bellman target.

16 Minimal Mental Model

The shortest way to remember Q-learning is:

$$Q(s, a) \leftarrow \text{reward now} + \text{best discounted value later.}$$

Once Q is accurate, the policy is easy:

choose the action with the largest Q-value.